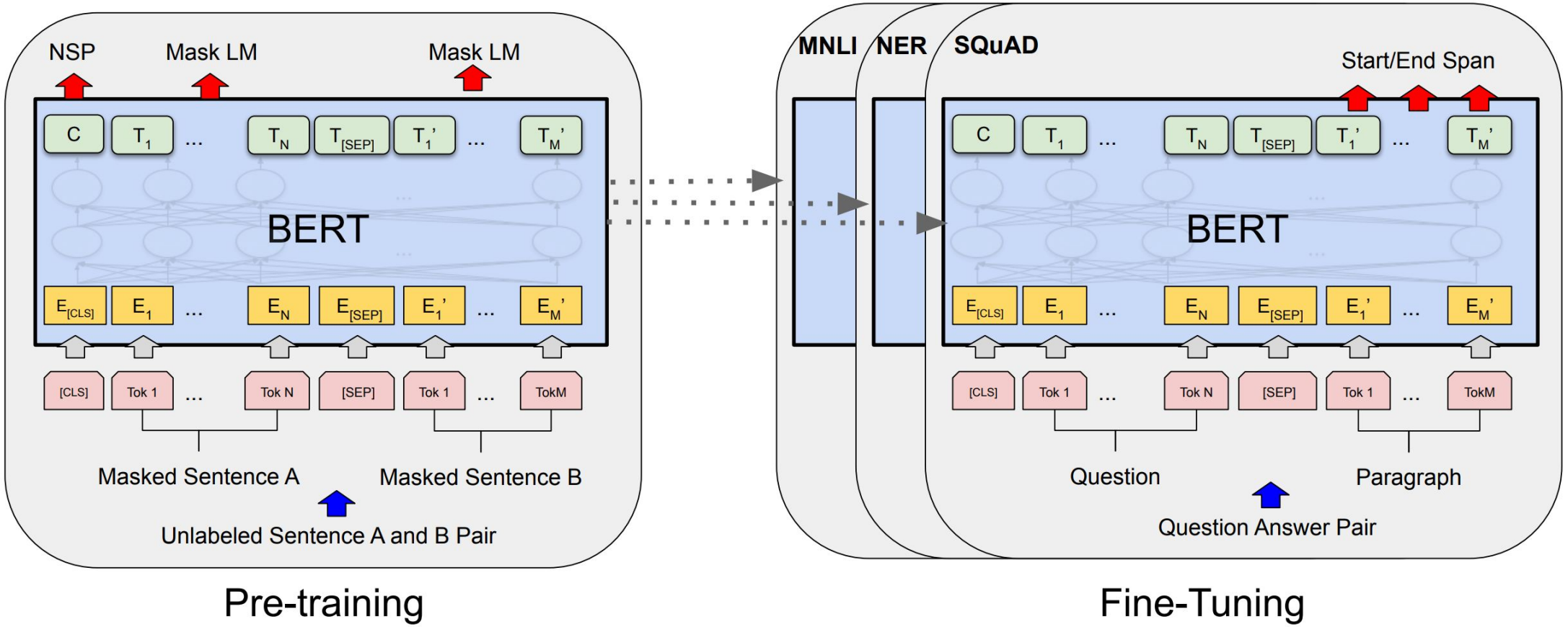
Language Model Fine-tuning and Adaptation

CS728
Soumen Chakrabarti

Pre-train, fine-tune paradigm



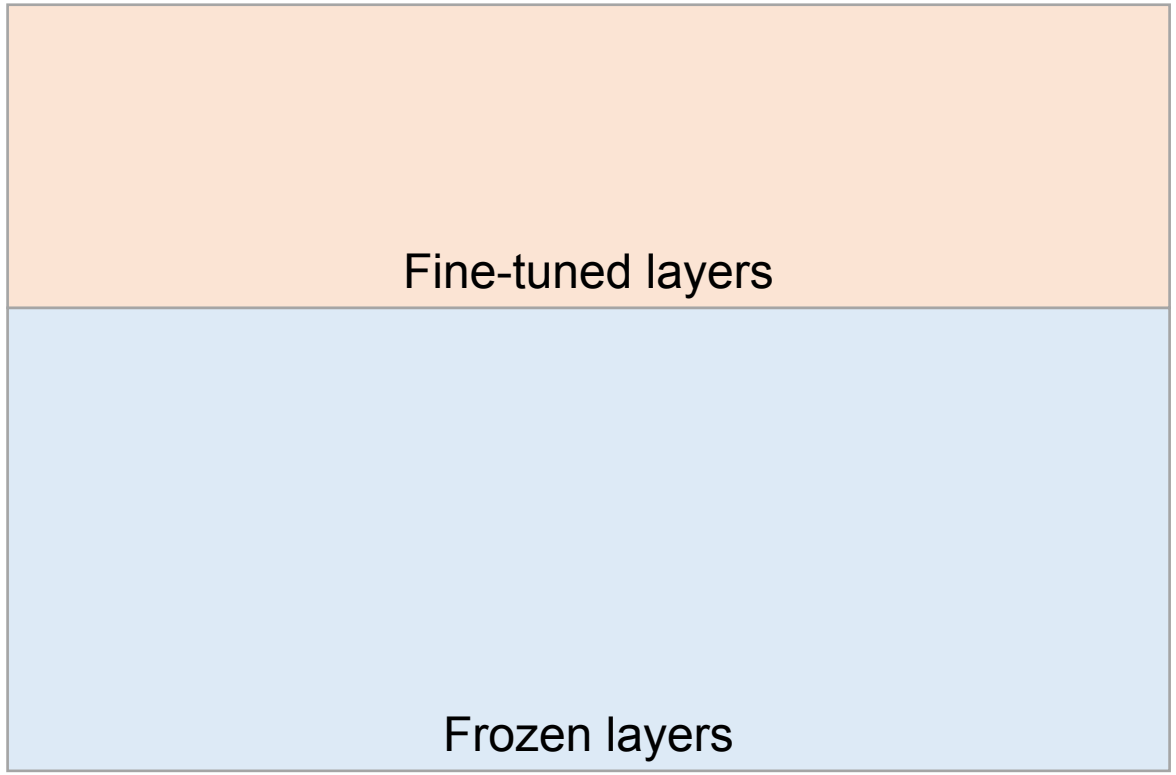
- NSP=next sentence prediction, +masked word
- Diversify pretrained model to multiple fine-tuned models, one for each task

Resource hungry LMs

- GPT-3 consumed around 1,287 MWh (running the London Eye continuously for over 5 years)
- GPT-3 training used ~355 GPU-years on V100s
- GPT-3 required ~ 700K liters of water (a million water bottles)
- Train-time savings
  - Adapters, prefix-tuning, prompt-tuning
  - Low rank re-parameterization
- Test-time savings
  - Long context support (also for training)
  - Caching computation

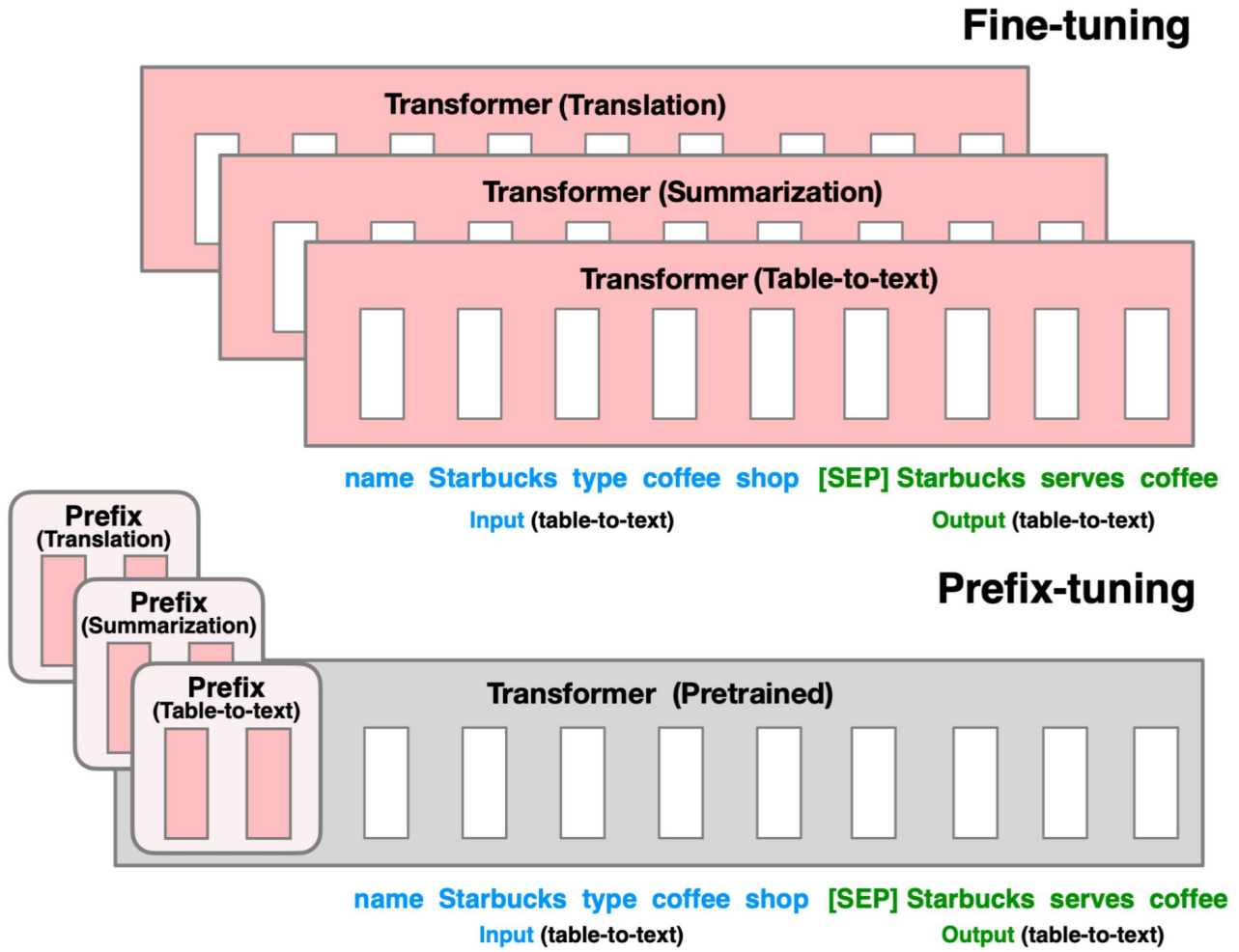
Cheapest and simplest: output layers

- Attach task-specific “classification heads” to output layer
  - Simple FF networks
- Train only these and freeze the transformer
- Or float the FF and last few layers of the transformer, freeze rest



Prefix tuning

- Alternative to layerwise separation of what's tuned vs not
- Output tokens z_i
- Autoregressive: [prefix; x; y]
- Encoder-decoder models: [prefix; x; prefix′; y]
- Transformer passes gradients through, but parameters not updated
- prefix, prefix′ tuned



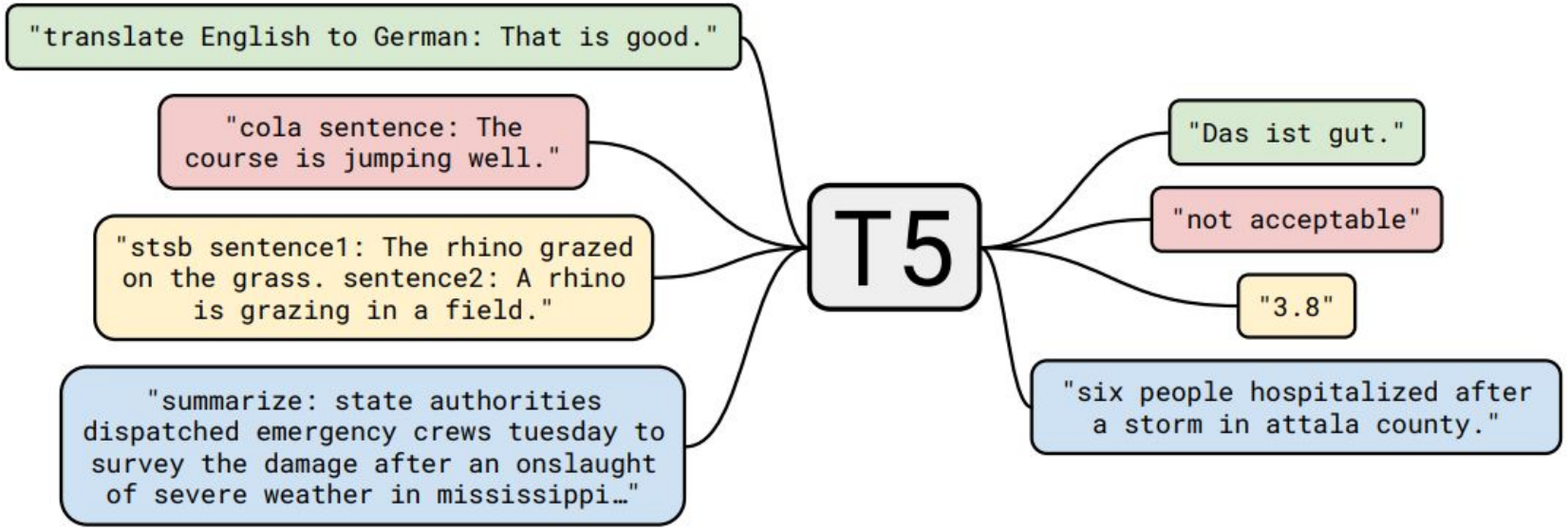
Prefix tuning, continued

- Training standard autoregressive LM:

$$\max_{\phi} \log p_{\phi}(y|x) = \sum_{i \in \text{Yidx}} \log p_{\phi}(z_i|h_{<i})$$

- Yidx are index of positions of output
- Pidx are index of positions of prefix
- Define $h_i = \begin{cases} P_{\theta}[i,:], & i \in \text{Pidx} \\ \text{LM}_{\phi}(z_i|h_{<i}), & \text{otherwise} \end{cases}$
- Here P_{θ} is the prefix matrix to be tuned; ϕ frozen
  - Note, more general than discrete token search
- Limit degrees of freedom, lift from smaller matrix

T5: text-to-text transfer tranformer



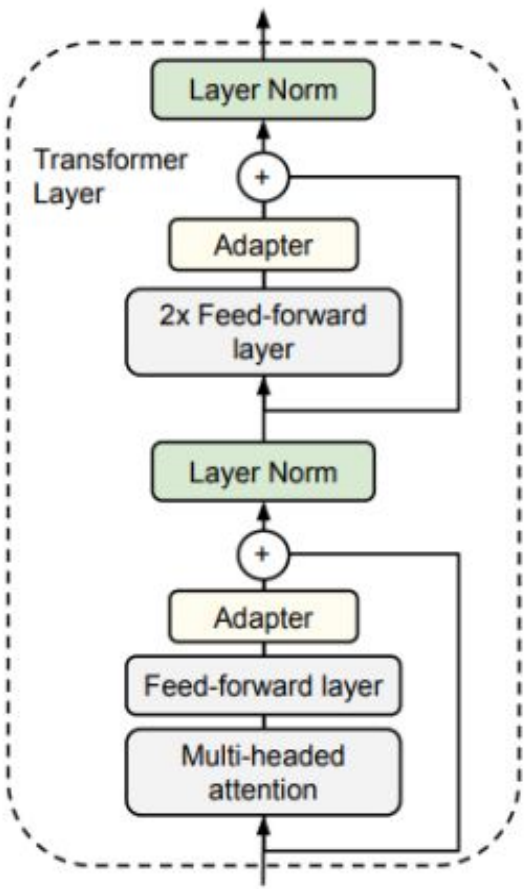
- Choose a fixed string “op code” for each role T5 used in
- T5 first pre-trained for usual NL objectives
- Then fine-tuned to encode [op code, input] and decode [output]
  - stsb = Semantic Textual Similarity Benchmark
- Probably first instance of replacing forked task-specific output network with instruction given in language to a single fine-tuned network

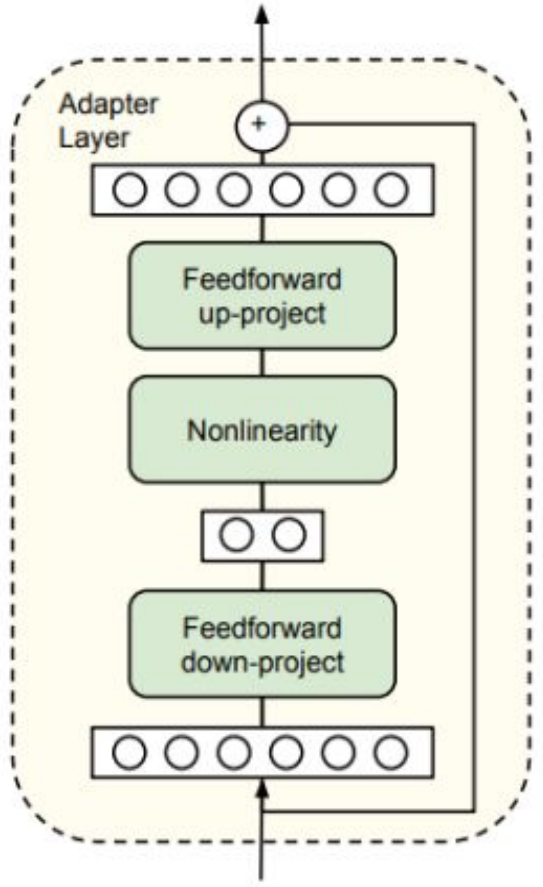
Dynamic selection of params to update

- ϕ are all params of LM, ϕ_i one param element
- $\left|\nabla_{\phi_i}(\text{loss})\right|$ is absolute sensitivity
- $\frac{\left|\nabla_{\phi_i}(\text{loss})\right|}{(|\phi_i|+\epsilon)^a}$ is a smoothed estimate of relative sensitivity (a is a hyperparameter)
- Order param elements by decreasing relative sensitivity and float only top ones
- Repeat selection “every now and then”
- Match full FT perf while tuning only 0.15% params

Pre-insertion of cheap adapter layers

- Showing encoder layer
- Insert two adapter blocks
- Each expanded to project down, nonlinear, project up
- Adapter can itself be bypassed
- Adapter blocks contain 3.6% of all params
- Train these, freeze rest
- 99.6% of full FT perf

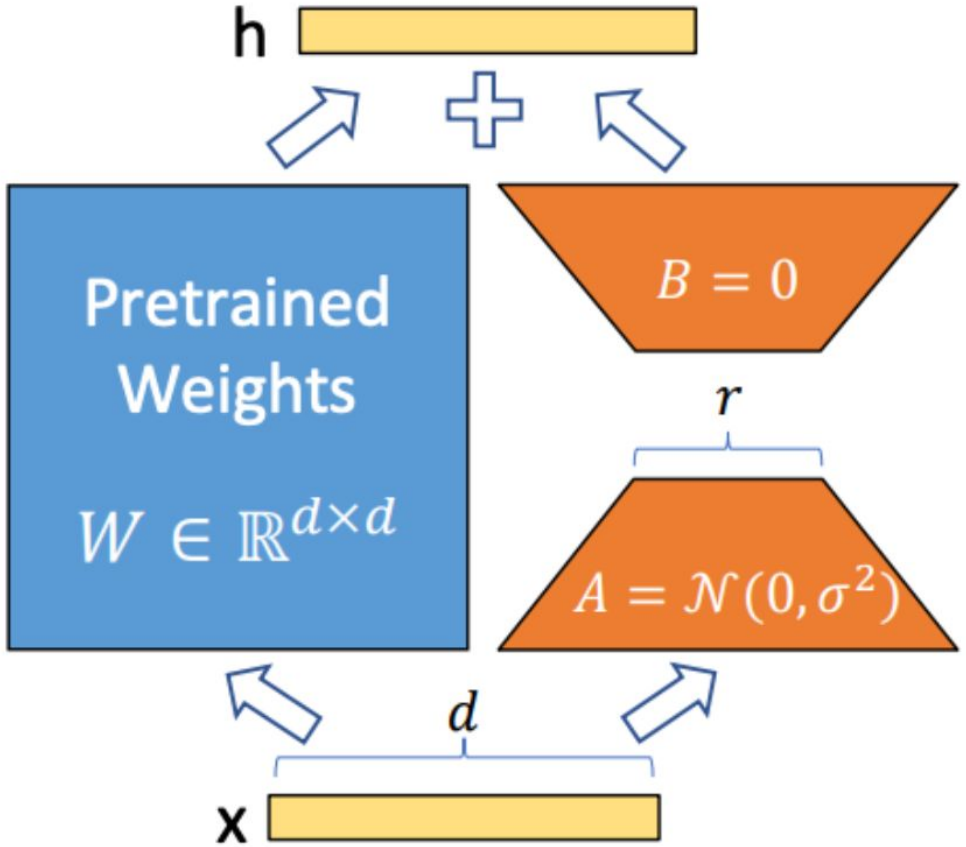




Have to plan for adaptation before pretraining, cannot be an ‘afterthought’

Reparameterization

- Linear layers in various places of a transformer
  - W_Q, W_K, W_V in attention
  - FF network after attention
  - W_O to aggregate heads
- If FT amounts to moving W_0 to $W_0 + \Delta W$, model ΔW as a low-rank matrix $\Delta W = AB$
  - Hold W_0 fixed
  - Fine tune only A and B

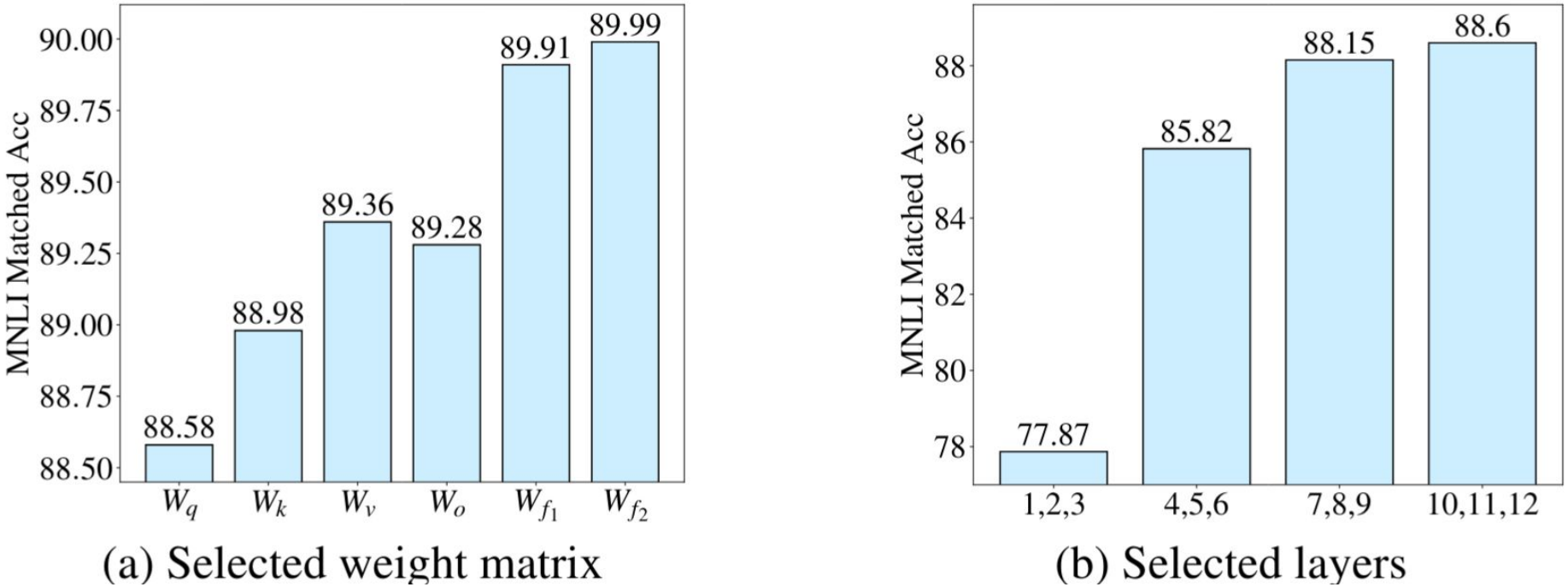


$$h = xW$$
$$= x(W_0 + AB)$$
$$= xW_0 + xAB$$

LoRA advantages and limitations

- Similar to BERT diversification, but very few parameters stored per diversified model
- Switch tasks by simply replacing A and B matrices
- Can work with very large models
- Sensitive to hyperparameters
- Careful rank selection needed
  - Best rank choice may not be fixed across various linear layers in a transformer
  - May even limit to specific layers or W matrices

AdaLoRA



- W_{f_1}, W_{f_2} in FF layers matter more than $W_{q,k,v}$
- Later layers matter more than early layers
- Given a sparsity budget/constraint, how to choose matrices and ranks?

Direct SVD emulation

- Let $W_k = W_k^0 + \Delta_k = W_k^0 + P_k \Lambda_k Q_k$ be the intended low rank correction to W_k^0 to FT W_k
- $\Lambda_k \geq 0$ is easy to enforce: $\Lambda_k = \left[\widetilde{\Lambda_k}\right]_+$
- Regularize P_k, Q_k 'toward' orthogonality
  - $\Omega(P_k, Q_k) = \left\| P_k^\top P_k - \mathbb{I} \right\|_F^2 + \left\| Q_k Q_k^\top - \mathbb{I} \right\|_F^2$
  - Get overall loss $\mathcal{L}$ after adding $\gamma \sum_k \Omega(P_k, Q_k)$
- Sample update at time n

$$\Lambda_k^{(n+1/2)} = \Lambda_k^{(n)} - \eta \nabla_{\Lambda_k} \mathcal{L}(\cdots)$$

Pruning by ranks

- With each P_k, Λ_k, Q_k we will associate a sensitivity or importance score S_k^n
- $\Lambda_k^{(n+1)} \leftarrow \Lambda_k^{(n+1/2)}$ if importance is among the top, otherwise $\Lambda_k^{(n+1)} \leftarrow 0$ (reduce rank)
- Various heuristics to compute sensitivity based on gradients and singular values
- While elements of Λ_k keep getting zeroed out, P_k, Q_k are maintained in full to allow 'comebacks'

What is model distillation?

- Best general-purpose LMs are large and expensive
- Specific applications may not need so much power
- Training application-specific LMs is also expensive
- Parameter-efficient fine-tuning is one solution
  - Reduces training but not inference time
  - Composite model not compact for edge devices
- Alternative: use pretrained or fine-tuned large LM as teacher and small LM as student
  - Choices of optimization objective

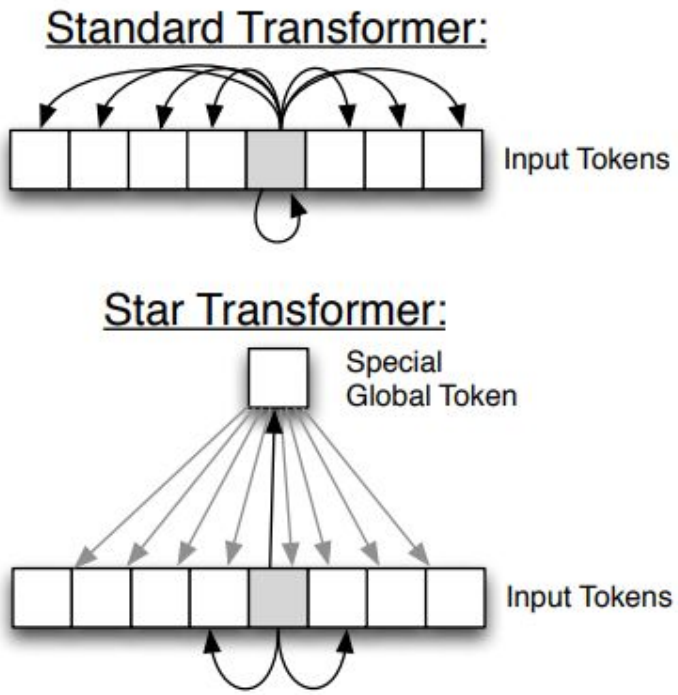
All-to-all attention bottleneck

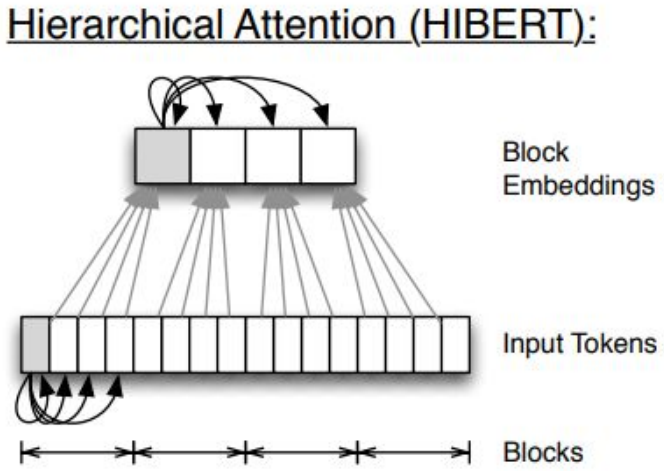
- Attention from each token position i to each token position j is computed in each layer
- Bad enough for encoder-only
- Gets longer and longer as decoder works
- How to efficiently support long contexts?
  - Forcibly sparsify interactions
  - Cache computation results to reuse (KV cache)

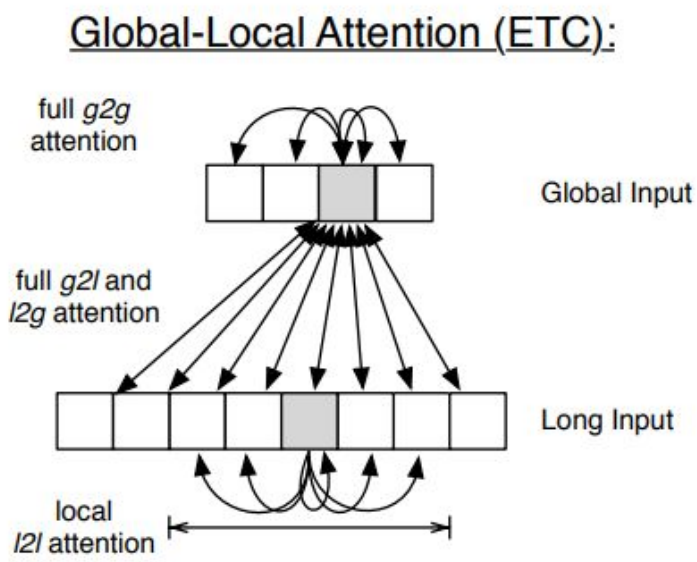
Dealing with long contexts

- Reformer (2020/Jan/13)
  - Chunked matrix-vector product with less GPU RAM
  - Reduce all-to-all attention cost using LSH
- ETC (2020/Apr)
  - Hierarchy, local, global attention patterns
- Random feature attention (2021/Mar)
  - A form of random hashing
- LongFormer (2020/Apr/10)
  - Local window + task-driven global attentions
  - Overall attention cost is sparse and linear
- BigBird (2020/Jul/28)
  - Another sparse attention design
  - Overlay of band-diagonal, global, random

ETC

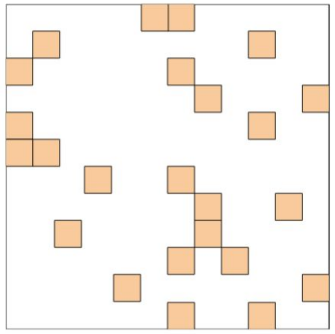




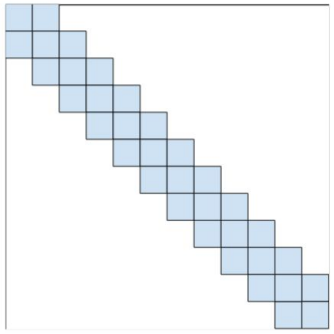


- Relative positional embeddings
- Local attention
- Hierarchy for global signals
- Relations between pairs of tokens

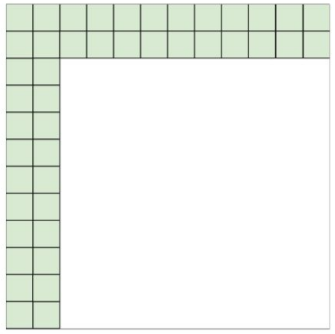
Attention stencils for long contexts



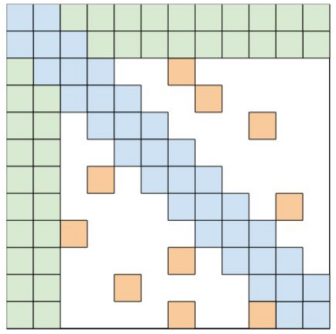
(a) Random attention



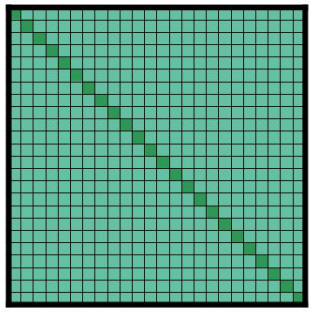
(b) Window attention



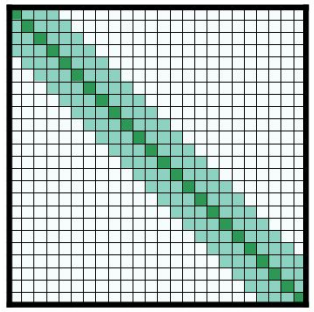
(c) Global Attention



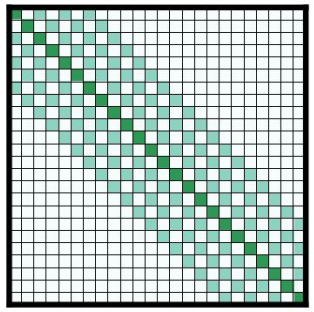
(d) BIGBIRD



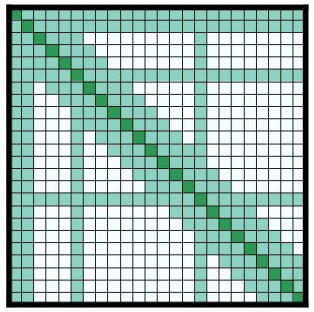
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window



(d) Global+sliding window

- Exploit sparse patterns from tensor-optimized code
- Cannot have too many random connections

Naive “decoder only” invocation

- Can design a LM ‘API’ as a function call that outputs one token: $x_{t+1} = f(x_{1:t})$
  - $f(\text{“Mary had”}) \rightarrow \text{“a”}$; $f(\text{“Mary had a”}) \rightarrow \text{“fat”}$; $f(\text{“Mary had a fat”}) \rightarrow \text{“pig”}$
- Lot of repeated redundant computation
- We want storage and computation to be shared across successive invocations
- Token at t does not affect $\boldsymbol{x}_{<t}^{(0:L)}, \boldsymbol{k}_{<t}^{(0:L)}, \boldsymbol{v}_{<t}^{(0:L)}$

Illustration of repeated computation

